

Lecture Notes on Pointers in C

Mohammad Maksood Akhter, Ph.D.

Assistant Professor

Department of Computer Science and Engineering

Abstract

These notes cover the fundamental and advanced concepts of **Pointers in C**. It explains pointer basics, pointer arithmetic, type conversions, parameter passing, pointer-to-pointer, and pointer to arrays, with examples and intuitive memory diagrams.

I. INTRODUCTION TO POINTERS

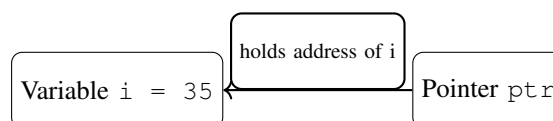
A **pointer** is a variable that stores the *address* of another variable. Two important operators are used with pointers:

- **&** (Address-of operator): gives the address of a variable.
- ***** (Dereference or Indirection operator): gives the value stored at the address.

```

1  #include <stdio.h>
2  int main() {
3      int i = 25;
4      int *ptr;
5      ptr = &i;
6      printf("Address of i = %p\n", &i);
7      printf("Address stored in ptr = %p\n", ptr);
8      printf("Value of i = %d\n", i);
9      printf("Value pointed by ptr = %d\n", *ptr);
10     *ptr = 35;
11     printf("Modified value of i = %d\n", i);
12     return 0;
13 }
```

Output: Address of i = 0x7ffe2b...
 Address stored in ptr = 0x7ffe2b...
 Value of i = 25
 Value pointed by ptr = 25
 Modified value of i = 35



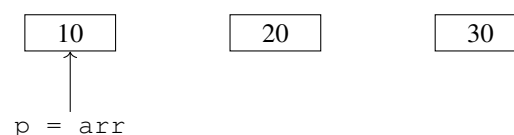
Pointer size depends on system architecture (e.g., 4 bytes on 32-bit, 8 bytes on 64-bit systems).

II. POINTER ARITHMETIC

```

1  int arr[3] = {10, 20, 30};
2  int *p = arr;
3  printf("%d ", *p);    // 10
4  p++;
5  printf("%d ", *p);    // 20
```

When you increment a pointer ($p++$), it moves to the next element according to its data type size.



III. POINTER TYPE CONVERSION

```

1  int i = 265;
2  char *ptr;
3  ptr = (char *)&i;
4  int j = *(int *)ptr;
5  printf("%d", j);

```

If you store an `int*` address in a `char*`, only one byte is accessed. To correctly retrieve the integer, you must **typecast back** using `(int*)`.

IV. PARAMETER PASSING TECHNIQUES

Call by Value

```

1  void swap(int a, int b) {
2      int t = a;
3      a = b;
4      b = t;
5  }
6  int main() {
7      int x = 10, y = 20;
8      swap(x, y);
9      printf("%d %d", x, y);
10 }

```

Output: 10 20

Call by Reference

```

1  void swap(int *a, int *b) {
2      int t = *a;
3      *a = *b;
4      *b = t;
5  }
6  int main() {
7      int x = 10, y = 20;
8      swap(&x, &y);
9      printf("%d %d", x, y);
10 }

```

Output: 20 10

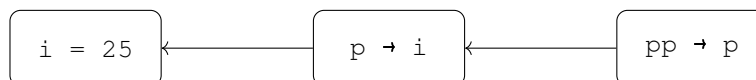
In call by reference, actual parameters are modified because addresses are passed to the function.

V. POINTER TO POINTER

```

1  int i = 25;
2  int *p = &i;
3  int **pp = &p;
4  printf("%d", **pp);

```



VI. POINTER TO ARRAY

```

1  int a[5] = {10, 20, 30, 40, 50};
2  int *ptr = a;
3  printf("%d ", *ptr); // 10
4  printf("%d ", *(ptr+2)); // 30

```

Array name acts as a **constant pointer** to its first element. You cannot modify it (e.g., `a++` is invalid), but pointer variables like `ptr` can move through the array.

VII. POINTER SUBTRACTION EXAMPLE

```
1  int a[5] = {10,20,30,40,50};  
2  int *p1 = &a[4];  
3  int *p2 = &a[0];  
4  printf("%ld", p1 - p2);
```

Output: 4

Subtracting two pointers gives the number of elements between them, not the byte difference.

VIII. SUMMARY

Pointer Type	Purpose	Example
Basic Pointer	Stores address of a variable	int *p;
Pointer to Pointer	Stores address of another pointer	int **pp;
Pointer to Array	Points to first element of array	int *p = a;
Pointer to Function	Holds address of a function	int (*fp) ();
Void Pointer	Generic pointer, typecast required	void *vp;

Pointers are powerful but must be used carefully — improper use can lead to segmentation faults or memory corruption.

END OF NOTES